



Database Languages

Software System Development Seminar

Wrocław University of Science and Technology

Department of Computer Science - Álvaro Puebla Ruisánchez - 2025 / 2026

Agenda

01 Introduction

What are database languages and why do they matter?

02 DBMS & DB Languages

Relationship between databases, DBMS, and languages

03 Language Categories

DDL · DML · DCL · TCL with SQL examples

04 SQL in Practice

The most widely used relational database language

05 NoSQL Query Languages

Modern non-relational approaches & MongoDB

06 Real-World Use Cases

Backend, analytics, web & enterprise applications

07 Summary & References

Key takeaways and further reading

Introduction: What are Database Languages?



Definition

A database language is a formal language used to define, manipulate, control, and query data stored in a database management system (DBMS).



Role in Software Systems

They serve as the primary interface between application code and persistent data - enabling create, read, update, and delete (CRUD) operations.



Standardisation

SQL (ISO/IEC 9075) is the dominant standard; NoSQL systems use proprietary or document-based query APIs.

Why Does It Matter?

~80%

of enterprise apps rely on a relational DB

SQL

has been the standard since the 1970s
(SEQUEL → SQL)

CRUD

operations are enabled entirely via DB languages

Cloud

data services (AWS RDS, Azure SQL) all expose SQL interfaces

DBMS Architecture: Where Database Languages Fit



Database

Organised collection of structured data

DBMS

Software that manages access to the database

DB Language

Formal syntax to interact with the DBMS

Categories of Database Languages: Overview

DDL

Data Definition Language

Defines and modifies the structure of the database schema (tables, indexes, views, constraints).

Key Commands

CREATE

ALTER

DROP

TRUNCATE

DML

Data Manipulation Language

Used to insert, retrieve, update, and delete records within existing database tables.

Key Commands

SELECT

INSERT

UPDATE

DELETE

DCL

Data Control Language

Controls access rights and permissions; defines who can perform which operations.

Key Commands

GRANT

REVOKE

TCL

Transaction Control Language

Manages database transactions to ensure data integrity and atomicity.

Key Commands

COMMIT

ROLLBACK

SAVEPOINT

DDL: Data Definition Language

Purpose

DDL statements define and modify the database schema - the logical structure of all tables, indexes, views, and constraints. They are used at design time and during schema migrations, and do not operate on data rows.

CREATE TABLE

Creates a new table with defined columns and data types

ALTER TABLE

Adds, modifies, or drops columns/constraints on existing tables

DROP TABLE

Permanently removes a table and all its data from the schema

TRUNCATE

Removes all rows from a table without logging individual row deletes



```
-- Create a students table
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    enrolled_on DATE DEFAULT CURRENT_DATE
);

-- Add a new column later
ALTER TABLE students
    ADD COLUMN gpa DECIMAL(3,2);

-- Remove the table entirely
DROP TABLE students;
```

DML: Data Manipulation Language (CRUD)

SELECT

Query and retrieve data - the most frequently used SQL statement

INSERT

Add new rows to a table

UPDATE

Modify existing rows matching a condition

DELETE

Remove rows matching a condition



```
-- SELECT: retrieve students with GPA > 3.5
SELECT name, email, gpa
FROM   students
WHERE  gpa > 3.5
ORDER  BY gpa DESC;

-- INSERT: add a new student
INSERT INTO students (student_id, name, email)
VALUES (101, 'Anna Kowalska', 'anna@pwr.edu.pl');

-- UPDATE: correct an email address
UPDATE students
SET   email = 'anna.kowalska@pwr.edu.pl'
WHERE student_id = 101;

-- DELETE: remove graduated students
DELETE FROM students
WHERE  enrolled_on < '2020-01-01';
```

DCL & TCL: Access Control and Transaction Management

DCL - Data Control Language

GRANT

Give a user or role specific privileges on a database object

REVOKE

Remove previously granted privileges from a user or role



```
-- Grant SELECT and INSERT to a user
GRANT SELECT, INSERT
  ON  students
  TO  'lecturer'@'localhost';

-- Remove INSERT privilege
REVOKE INSERT
  ON  students
  FROM 'lecturer'@'localhost';
```

TCL - Transaction Control Language

COMMIT

Permanently save all changes made in the current transaction

ROLLBACK

Undo all uncommitted changes and restore previous state

SAVEPOINT

Create a named checkpoint within a transaction to roll back to



```
-- Atomic bank transfer
BEGIN TRANSACTION;

  UPDATE accounts
    SET balance = balance - 500 WHERE account_id = 1;
  UPDATE accounts
    SET balance = balance + 500 WHERE account_id = 2;

COMMIT;    -- success: save all
ROLLBACK;  -- failure: undo all
```


SQL: Structured Query Language

1970

E.F. Codd's Relational Model (IBM)

ISO 9075

International Standard for SQL

SQL:2023

Latest revision of the SQL standard

Declarative

You specify WHAT to retrieve - the DBMS decides HOW

Standardised

ISO/IEC 9075; portable across PostgreSQL, MySQL, Oracle, SQL Server

4 Sub-languages

Encompasses DDL, DML, DCL, and TCL in one unified language

Powerful Queries

Supports JOINS, subqueries, aggregation, and window functions

ORM Integration

Embedded via JDBC, ODBC, SQLAlchemy, Hibernate, and more

Multi-table JOIN Example

```
-- Query three tables with JOIN
SELECT  s.name,
        c.course_name,
        e.grade
FROM    students      s
JOIN    enrolments    e
      ON e.student_id = s.student_id
JOIN    courses        c
      ON c.course_id  = e.course_id
WHERE   c.department = 'CS'
ORDER  BY e.grade DESC
LIMIT  10;
```

NoSQL: Query Languages in Non-Relational Databases

Beyond SQL: NoSQL Systems

NoSQL databases emerged in the 2000s to handle massive scale, flexible schemas, and diverse data models that relational databases struggled with.

Each NoSQL type uses its own query interface.

Document

MongoDB, Couchbase

JSON-style query API

Key-Value

Redis, DynamoDB

GET / SET / DEL commands

Column-Family

Cassandra, HBase

CQL (Cassandra Query Language)

Graph

Neo4j, Amazon Neptune

Cypher / SPARQL query language



```
// MongoDB Document Query API
```

```
// INSERT a document
db.students.insertOne({
  _id: 101,
  name: "Anna Kowalska",
  email: "anna@pwr.edu.pl",
  courses: ["Databases", "Algorithms"]
});
```

```
// FIND students enrolled in Databases
db.students.find(
  { courses: "Databases" },
  { name: 1, email: 1 }
);
```

```
// UPDATE: push a new course
db.students.updateOne(
  { _id: 101 },
  { $push: { courses: "Machine Learning" } }
);
```

SQL vs. NoSQL: Comparative Overview

Feature	SQL (Relational)	NoSQL (e.g. MongoDB)
Data Model	Tables with rows and columns	Documents, key-value, graphs
Schema	Fixed, enforced at design time	Flexible / schema-less
Query Language	Standardised SQL (ISO)	Proprietary API / CQL / Cypher
Transactions	Full ACID support	Eventual or BASE consistency
Scalability	Vertical (scale-up)	Horizontal (scale-out)
Relationships	Foreign keys, JOINS	Embedded documents / references
Best for	Banking, ERP, reporting	IoT, social media, real-time apps
Examples	PostgreSQL, MySQL, Oracle	MongoDB, Redis, Cassandra

💡 Choose based on your data structure, consistency needs, and scale requirements - not fashion.

Real-World Use Cases of Database Languages



Backend Development

REST APIs use SQL/ORM to persist user data, session state, and business entities. Frameworks: Django ORM, Hibernate, Entity Framework.



Data Analytics & BI

Analysts write complex SQL queries against data warehouses (Snowflake, BigQuery) to generate dashboards and business reports.



Web Applications

E-commerce platforms (product catalogues, orders) and social networks (user profiles, feeds) rely on both SQL and NoSQL databases.



Enterprise Systems

ERP and CRM systems (SAP, Salesforce) use SQL databases for financial records, inventory, and customer data with strict ACID guarantees.

Key Takeaways

01

Database languages are essential interfaces

They define, manipulate, control, and transact data in any DBMS - without them, no application can persist state.

02

SQL is structured into four sub-languages

DDL (schema), DML (data), DCL (access), TCL (transactions) - each with distinct commands and purposes.

03

SQL is declarative and widely standardised

ISO SQL is supported by all major RDBMS platforms; knowing SQL is transferable across database systems.

04

NoSQL offers flexibility for modern data

Document, key-value, column, and graph databases use their own query APIs optimised for scale and schema flexibility.

05

Choose the right tool for the right problem

SQL for structured, relational, transactional data. NoSQL for flexible, scalable, high-volume scenarios.

References

- [1] Ramez Elmasri & Shamkant B. Navathe - Fundamentals of Database Systems, 7th ed., Pearson, 2016.

- [2] Raghu Ramakrishnan & Johannes Gehrke - Database Management Systems, 3rd ed., McGraw-Hill, 2002.

- [3] ISO/IEC 9075:2023 - Information Technology - Database Languages SQL. International Organization for Standardization, 2023.

- [4] PostgreSQL Documentation - SQL Commands. PostgreSQL Global Development Group, 2024. <https://www.postgresql.org/docs/>

- [5] MongoDB Documentation - Query Documents. MongoDB Inc., 2024. <https://www.mongodb.com/docs/>

- [6] Martin Fowler - NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley, 2012.

- [7] E.F. Codd - A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, Vol. 13, No. 6, 1970.

- [8] W3Schools SQL Tutorial - <https://www.w3schools.com/sql/> (introductory reference, supplemental).